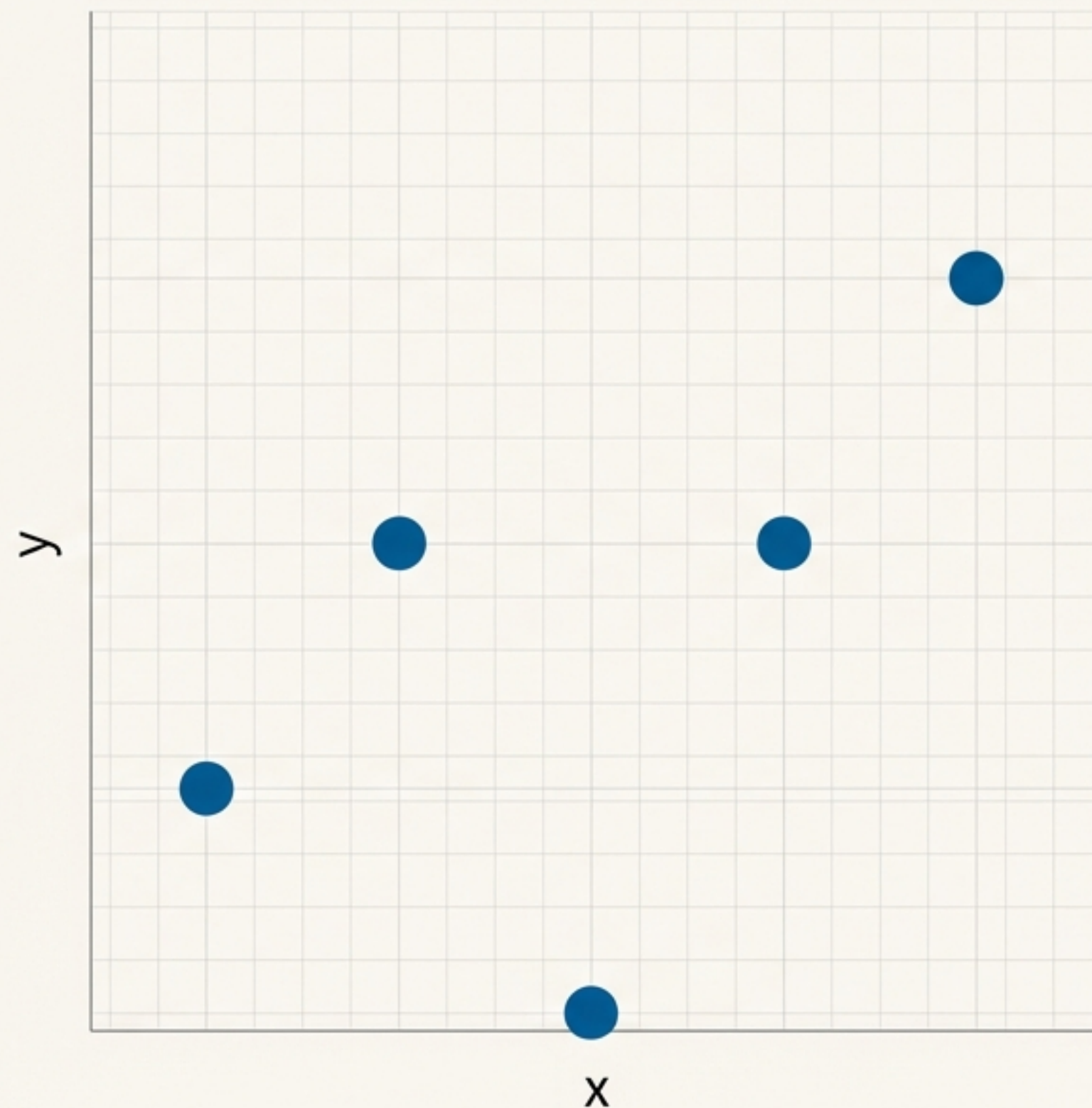


El Desafío de los Datos Discretos

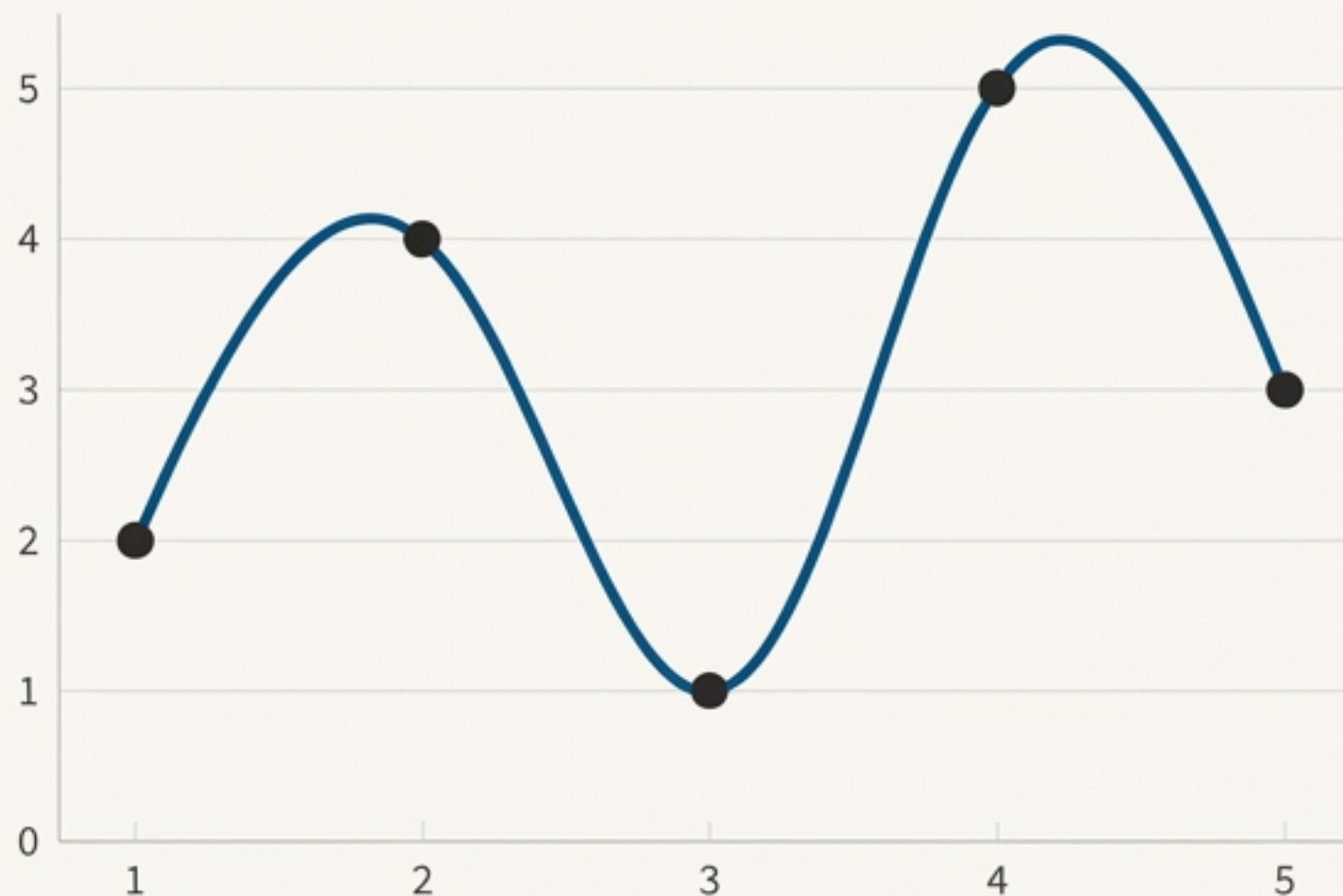
Imagina que estos son los resultados de un experimento. Tenemos mediciones en puntos específicos, pero... ¿qué sucede entre ellos? ¿Cómo podemos predecir el comportamiento del sistema en cualquier punto intermedio?

Necesitamos una forma de construir un modelo continuo a partir de datos discretos. Este es uno de los problemas fundamentales en modelado y simulación.



La Solución: El Polinomio Interpolante de Lagrange

El Polinomio Interpolante de Lagrange es una poderosa técnica matemática que construye un polinomio único que pasa *exactamente* por cada uno de nuestros puntos de datos. No es una aproximación general; es un ajuste perfecto a los datos que tenemos.



$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

La idea es construir el polinomio final como una suma ponderada, donde cada punto ' y_i ' aporta su 'influencia' a través de un polinomio base especial, ' $L_i(x)$ '.

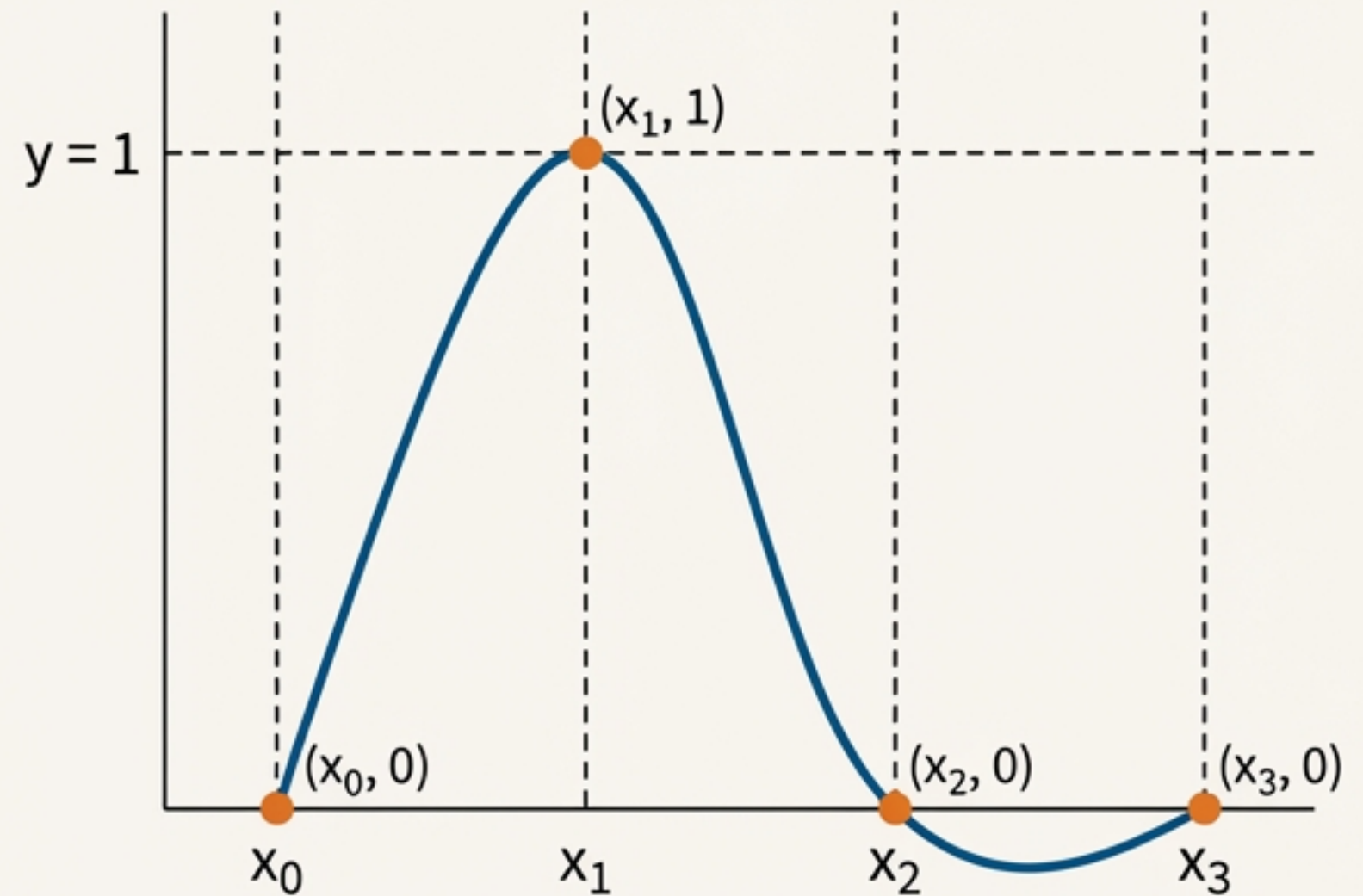
El Secreto: ¿Cómo Funciona el Polinomio Base $L_i(x)$?

La clave del método de Lagrange reside en la construcción de los polinomios base $L_i(x)$. Cada $L_i(x)$ está diseñado con un propósito muy específico:

- ✓ Toma el valor de **1** en el punto x_i .
- ✓ Toma el valor de **0** en todos los demás puntos x_j (donde $j \neq i$).

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Ejemplo Visual: $L_1(x)$ para 4 puntos



La Garantía Matemática: Existencia y Unicidad

Existencia

Siempre es posible construir un polinomio de interpolación de Lagrange para cualquier conjunto de puntos distintos (x_i, y_i) .

Unicidad

Dado un conjunto de $(n+1)$ puntos distintos, existe un **único** polinomio de grado n que pasa exactamente por todos esos puntos. El Teorema Fundamental de la Interpolación lo garantiza. Esto significa que no hay ambigüedad: la solución encontrada es **la** solución.



La Brecha con la Realidad: Analizando el Error de Interpolación

El polinomio $P(x)$ coincide perfectamente con nuestra función $f(x)$ en los puntos de datos. Pero, ¿cuál es el error en cualquier otro punto x ? La diferencia entre la función real y nuestro polinomio se puede expresar formalmente.

$$f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

$$f^{(n+1)}(\xi)$$

La $(n+1)$ -ésima derivada de la función real, evaluada en un punto desconocido ξ dentro del intervalo de nuestros datos.

$$\prod (x - x_i)$$

Este término depende de qué tan lejos está x de nuestros puntos de datos. El error es cero en los puntos de interpolación, como es de esperar.

De la Teoría a la Práctica: Calculando una Cota para el Error

Dado que no conocemos el valor exacto de ξ , no podemos calcular el error exacto. Sin embargo, podemos calcular una **cota superior** para el error, que es extremadamente útil en la práctica.

$$|E(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^n (x - x_i) \right|$$

donde:

$$M_{n+1} = \max_{\xi \in [x_0, x_n]} |f^{(n+1)}(\xi)| \quad (\text{El valor máximo de la } (n+1)\text{-ésima derivada en el intervalo})$$

- 1. Construir** el polinomio $P(x)$.
- 2. Calcular** el término del producto $\prod (x - x_i)$.
- 3. Determinar** el valor máximo M de la derivada.
- 4. Aplicar** la fórmula de la cota para encontrar el error máximo.

La Implementación: Del Concepto al Código en Python

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def polinomio_lagrange(x, x_puntos, y_puntos):
```

```
    n = len(x_puntos)
```

```
    P = 0
```

```
    for i in range(n):
```

```
        # Calcular el polinomio base de Lagrange
```

```
        li = 1
```

```
        for j in range(n):
```

```
            if i != j:
```

```
                li *= (x - x_puntos[j]) / (x_puntos[i] - x_puntos[j])
```

```
        P += y_puntos[i] * li
```

```
    return P
```

Aquí se construye
cada $L_i(x)$.

Aquí se realiza la
suma ponderada
 $\sum y_i * L_i(x)$.

El Resultado: Visualización del Polinomio Interpolante

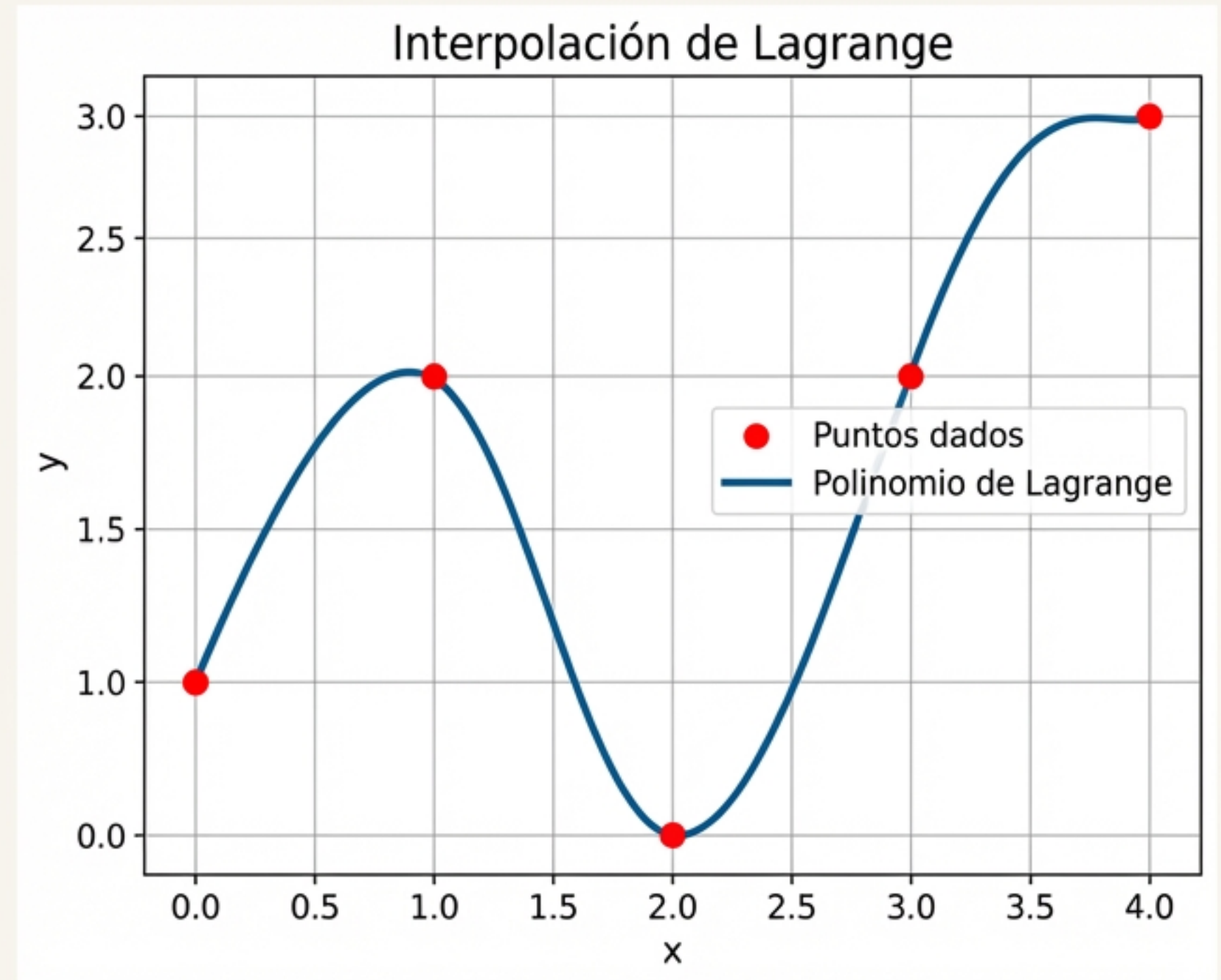
Datos de Entrada:

```
x_puntos = np.array([0, 1, 2, 3, 4])  
y_puntos = np.array([1, 2, 0, 2, 3])
```

Polinomio Reconstruido:

El polinomio de Lagrange resultante es:

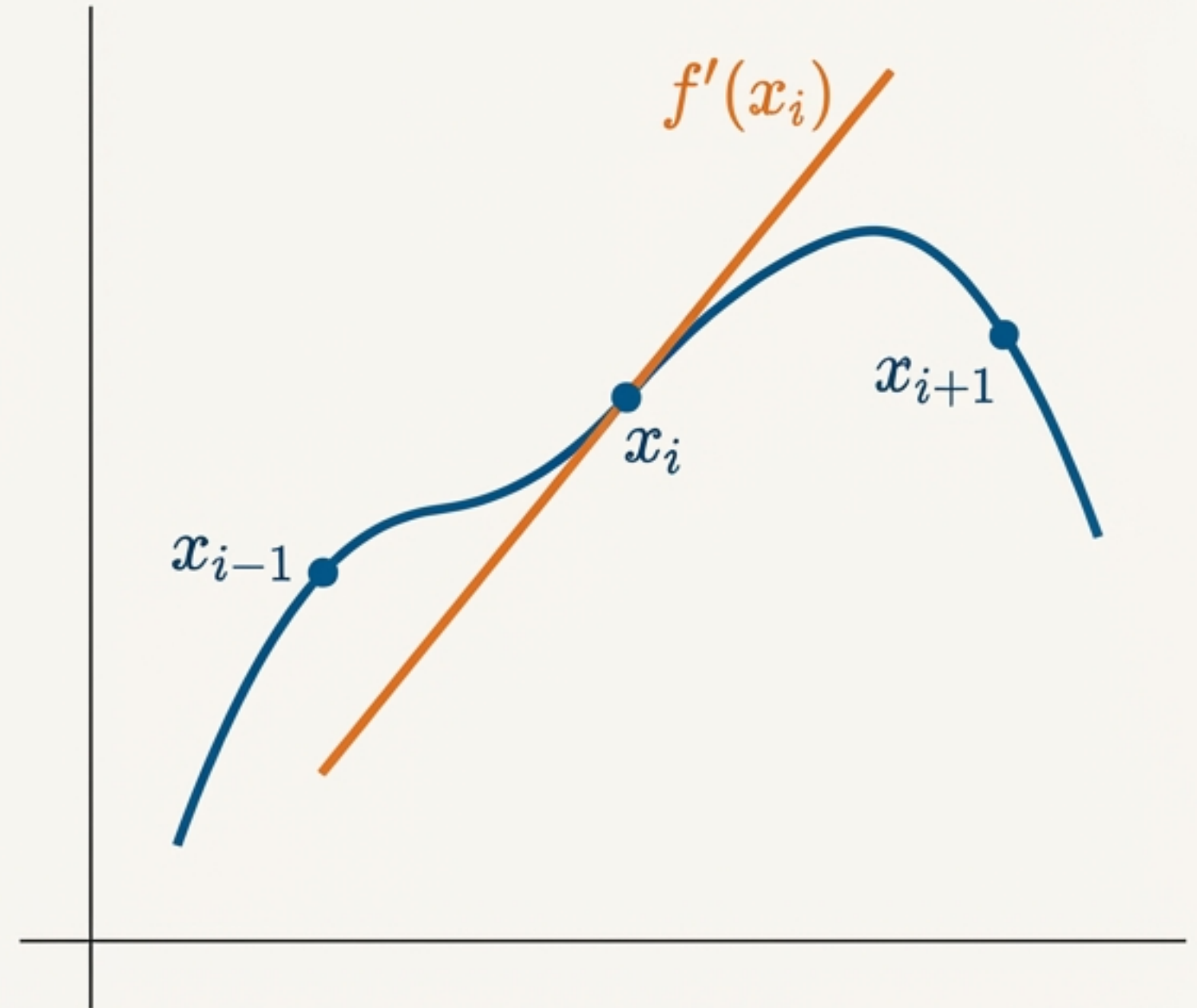
$$P(x) = 0.5833x^4 - 4.5x^3 + 11.417x^2 - 9.5x + 1$$



Un Nuevo Desafío: ¿Y la Tasa de Cambio?

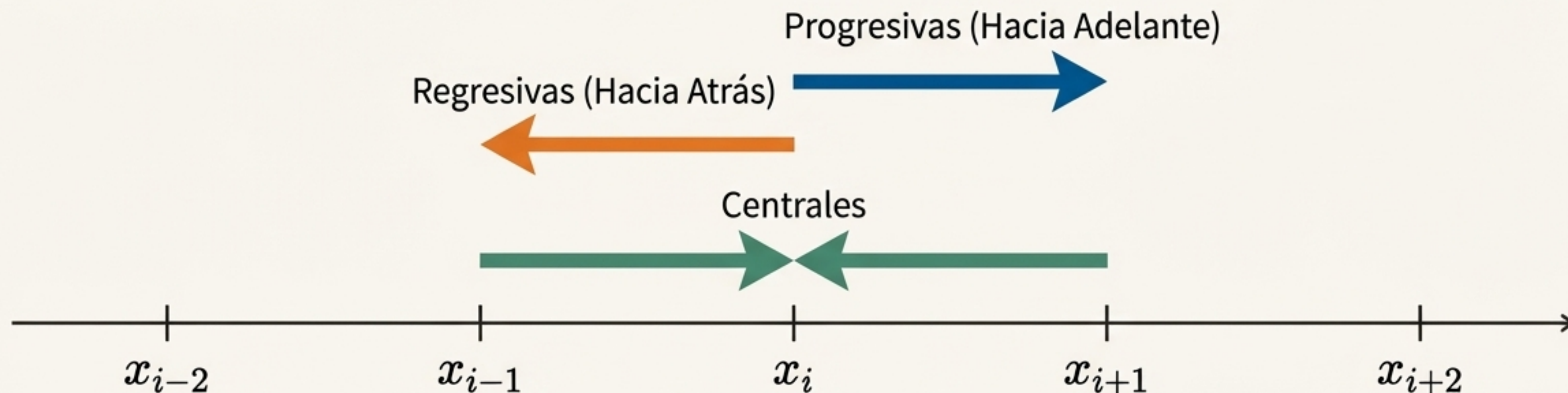
Hemos modelado exitosamente la función que se ajusta a nuestros datos. Pero en ingeniería y ciencia, a menudo no solo nos importa el valor, sino también su tasa de cambio: **la derivada.**

¿Cómo podemos aproximar la derivada de una función si solo conocemos su valor en puntos discretos?



La Herramienta para Derivadas: Diferencias Divididas

Las diferencias divididas (o diferencias finitas) son una forma de aproximar derivadas utilizando los valores de la función en puntos cercanos. Dependiendo de los puntos que usemos, tenemos tres enfoques principales:

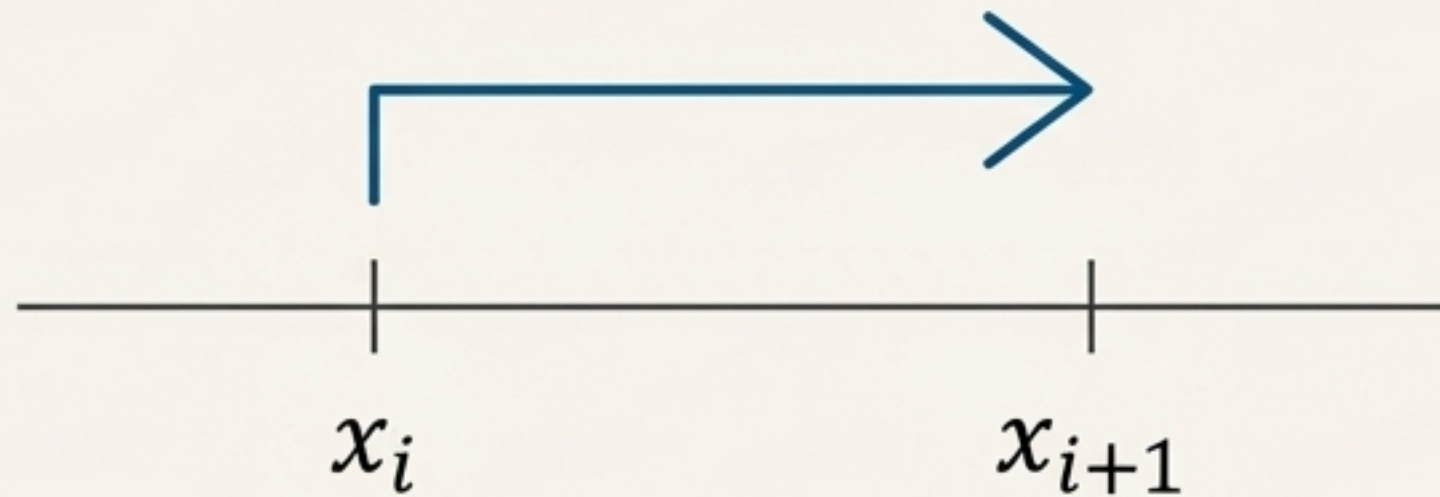


1. **Progresivas (Hacia Adelante):** Usan el punto actual y puntos futuros.
2. **Regresivas (Hacia Atrás):** Usan el punto actual y puntos pasados.
3. **Centrales:** Usan puntos simétricamente alrededor del punto actual. A menudo, son las más precisas.

Enfoque 1: Diferencias Divididas Progresivas

Este método aproxima la derivada en x_i utilizando la información de x_i y el siguiente punto, x_{i+1} . Es como mirar hacia el futuro inmediato.

(Nota: h es el paso constante entre puntos, $h = x_{i+1} - x_i$)



Primera Derivada

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h}$$

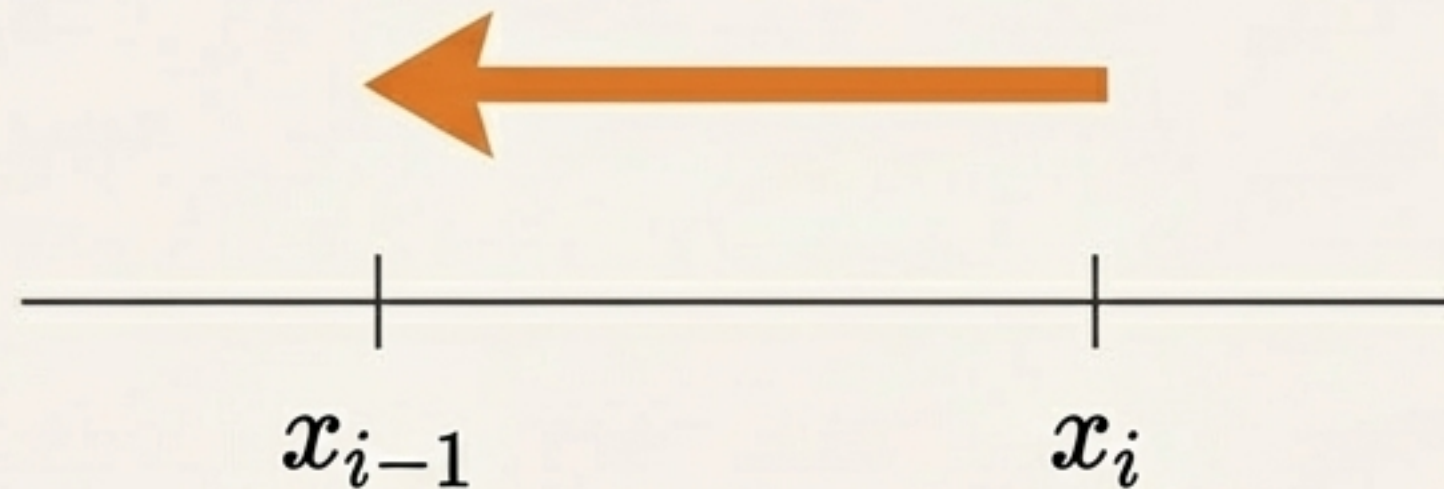
Segunda Derivada

$$f''(x_i) \approx \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$$

Enfoque 2: Diferencias Divididas Regresivas

Semejante al progresivo, pero en este caso, usamos el punto actual x_i y el punto anterior, x_{i-1} . Es como mirar al pasado inmediato para estimar el cambio.

(Nota: h es el paso constante entre puntos, $h = x_i - x_{i-1}$)



Primera Derivada

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{h}$$

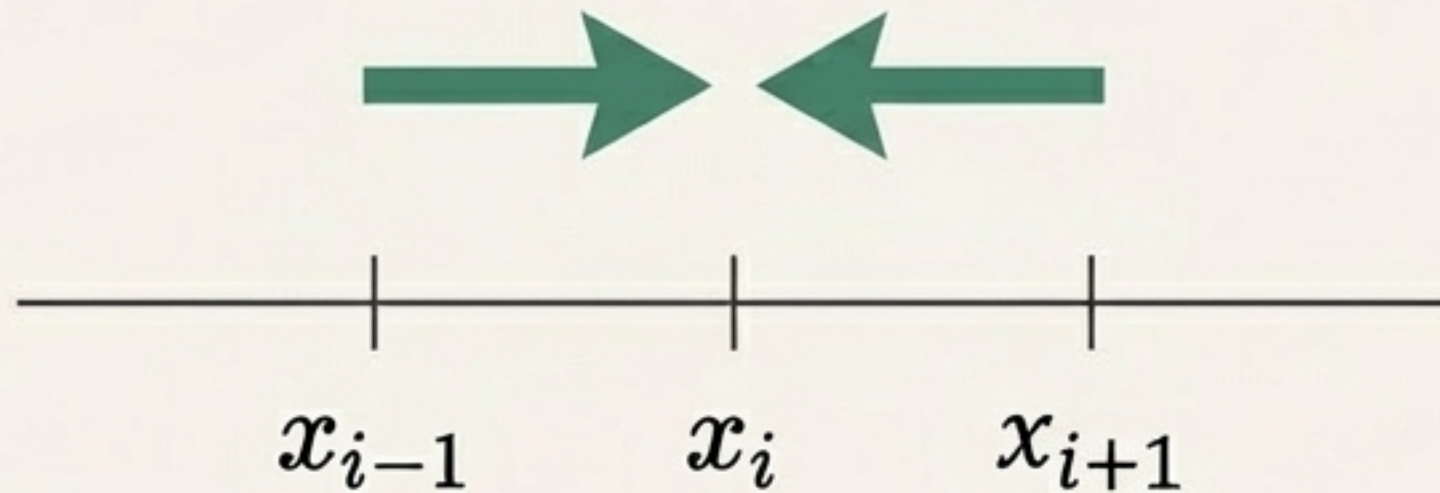
Segunda Derivada

$$f''(x_i) \approx \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2}))}{h^2}$$

Enfoque 3: Diferencias Divididas Centrales

Este método utiliza información de ambos lados de x_i (es decir, x_{i-1} y x_{i+1}), promediando efectivamente el cambio. Generalmente, esto conduce a una mejor aproximación del verdadero valor de la derivada.

(Nota: h es el paso constante entre puntos)



Primera Derivada

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1})}{2h}$$

Segunda Derivada

$$f''(x_i) \approx \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2}$$

Aplicación: Cálculo de Derivadas con Diferencias Centrales

Apliquemos esto a una función conocida para verificar su efectividad.

```
# Definir la función
def f(x):
    return x**3 - 2*x + 1

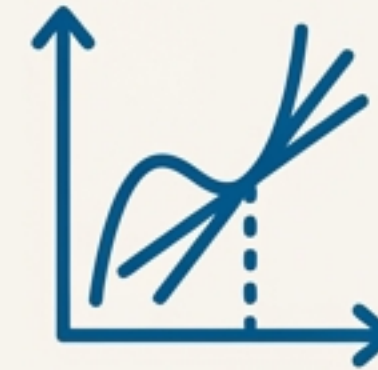
# Punto y paso
x = 2
h = 0.1

# Diferencias finitas centradas
def primera_derivada(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

def segunda_derivada(f, x, h):
    return (f(x + h) - 2*f(x) + f(x - h)) / (h**2)
```


La Solución Numérica

Función: $f(x) = x^3 - 2x + 1$
Punto de evaluación: $x = 2$
Paso: $h = 0.1$



Resultados del Script

Primera derivada en $x = 2$: **10.01**

Segunda derivada en $x = 2$: **12.0000000000000002**

Con unas pocas líneas de código, hemos obtenido excelentes aproximaciones para la primera y segunda derivada ($f'(2)=10$, $f''(2)=12$). Las diferencias divididas nos dan un método robusto y eficiente para analizar la dinámica de un sistema a partir de sus datos discretos, completando nuestro conjunto de herramientas de modelado.

